



**Waiting for everyone
to join**

**TU257
Information Systems
Marisa Llorens**

Week 11

IS

TU257

Agenda

- Regex
- Views
- Subqueries

Regular Expressions

REGULAR EXPRESSIONS

- Regular expressions are patterns used to match character combinations in strings.

`REGEXP_LIKE(expression, pattern)`

- Returns 1 if the string expression matches the regular expression specified by the pattern, 0 otherwise. If expression or pattern is NULL, the return value is NULL.

REGULAR EXPRESSIONS

- MySQL

REGEXP_LIKE(name, '^R')

- SQLite

name REGEXP '^R'

REGULAR EXPRESSIONS

- Match type:
 - c case sensitive
 - i case insensitive

`REGEXP_LIKE(name, '^R',i)`

REGULAR EXPRESSIONS

Metacharacter	Behavior
<code>^</code>	matches the position at the beginning of the searched string
<code>\$</code>	matches the position at the end of the searched string
<code>.</code>	matches any single character
<code>[...]</code>	matches any character specified inside the square brackets
<code>[^...]</code>	matches any character not specified inside the square brackets
<code>p1 p2</code>	matches any of the patterns p1 or p2
<code>*</code>	matches the preceding character zero or more times
<code>+</code>	matches preceding character one or more times
<code>{n}</code>	matches n number of instances of the preceding character
<code>{m,n}</code>	matches from m to n number of instances of the preceding character

REGULAR EXPRESSIONS MySQL

Find all contacts whose last names start with character A.

```
SELECT
    last_name
FROM
    contacts
WHERE
    REGEXP_LIKE(last_name, '^A');
```

Find all contacts whose last names finish with character n.

```
SELECT
    last_name
FROM
    contacts
WHERE
    REGEXP_LIKE(last_name, 'n$');
```

REGULAR EXPRESSIONS MySQL

Find all contacts whose last_name is either Anderson, Andersen, or Andersan.

```
SELECT
    last_name
FROM
    contacts
WHERE
    REGEXP_LIKE (last_name, 'Anders(o|e|a)n');
```

Find all products whose name starts with A, B or C.

```
SELECT    productname
FROM      products
WHERE
    REGEXP_LIKE(productname, '^(A|B|C)')
```

REGULAR EXPRESSIONS Case sensitivity

#first names that contain the letter c

REGEXP_LIKE(first_name, 'c', 'c')

#last names starting with the letter A or a

REGEXP_LIKE(last_name, '^a', 'i');

#first names that contain exactly two letters L or l

REGEXP_LIKE(first_name, 'l{2}', 'i')

Views

Creating **VIEWS**

- **VIEWS** are to a database what subroutines are to a programming language.
- A view is a named query stored in the database catalog.
- A view allows you to store a query in the database, so that you can access it later by name.

Creating **VIEWS**

- Views are treated in the same way as a normal table when writing queries.
 - e.g. write a query to tell you how many employees and how many projects are in the database. You could write them separately...

Open myCompany.db in
SQLite

Creating **VIEWS**

- Query finding how many employees and how many projects are in the database.
- Two queries:

```
SELECT COUNT(*) FROM employees;
```

```
SELECT COUNT(*) FROM projects;
```

Creating **VIEWS**(cont'd)

```
CREATE VIEW Summary(NumEmpl) AS SELECT COUNT(*) FROM employees;
```

```
CREATE VIEW Summary2(NumProj) AS SELECT COUNT(*) FROM
```

Name	Type	Schema
▼ Tables (3)		
> departments		CREATE TABLE "departments" ("DeptID" TEXT NOT NULL, "DeptName" TEXT NOT NULL)
> employees		CREATE TABLE "employees" ("employeeID" INTEGER NOT NULL, "employeeName" TEXT NOT NULL)
> projects		CREATE TABLE "projects" ("projectID" INTEGER NOT NULL, "projectName" TEXT NOT NULL)
▼ Indices (0)		
▼ Views (2)		
> Summary		CREATE VIEW Summary(NumEmpl) AS SELECT COUNT(*) FROM employees
> Summary2		CREATE VIEW Summary2(NumProj) AS SELECT COUNT(*) FROM projects
▼ Triggers (0)		

Creating **VIEWS**(cont'd)

```
SELECT NumEmpl, NumProj FROM Summary,  
Summary2;
```

	NumEmpl	NumProj
1	5	3

Advantages of using VIEWS

- Simplify complex query
 - Views help simplify complex queries. If you have any frequently used complex query, you can create a view based on it so that you can reference to the view by name instead of typing the query all over again.

Advantages of using VIEWS

- Make the business logic consistent
 - Suppose you have to repeatedly write the same formula in every query. Or you have a query that has complex business logic. To make this logic consistent across queries, you can use a view to store the calculation and hide the complexity.

Advantages of using VIEWS

- Add extra security layers
 - A table may expose a lot of data including sensitive data such as personal and banking information. By using views and privileges, you can limit which data users can access by exposing only the necessary data to them

Deleting **VIEWS**

- When you are finished with a **VIEW** you have created you can delete it, e.g.

DROP VIEW Summary;

DROP VIEW Summary2;

Sub-queries

Sub-queries

- A sub-query is a query nested inside another query.
- Sub-queries can return...
 - *a single value,*
 - *A row, or*
 - *A set of values*

Sub-queries

- SQL has two basic kinds of sub-queries
 - ***Derived*** table sub-queries
 - Appear in the **FROM** clause of a select statement
 - ***Expression*** sub-queries
 - Appear in the **WHERE** clause of a select statement
 - Sub-queries that return a single value or row
 - Sub-queries that are used to test a Boolean expression

Derived Table Sub-queries

- Allow you to list a query in the **FROM** clause of another query.
 - This allows you to create a temporary table and add it to the query, e.g.

```
SELECT address_id
```

```
FROM address_table
```

```
WHERE address_table.district ='Texas')
```

Derived Table Sub-queries

- Allow you to list a query in the **FROM** clause of another query.
- The query can be used within another query, e.g.

```
SELECT customers_table.email
```

```
FROM customers_table INNER JOIN (
```

```
    select address_id from address_table
```

```
    where address_table.district ='Texas') as texas
```

```
ON customers_table.address_id=texas.address_id
```

Derived Table Sub-queries

- Every derived table must have its own alias.
- The column names for a derived table come from its select list.

```
select customers_table.email  
  
from customers_table INNER JOIN (  
    select address_id from address_table  
    where address_table.district = 'Texas') as texas  
  
on customers_table.address_id=texas.address_id
```

address_id
10
122
310
405
567

texas table

Derived Table Sub-queries

ALTERNATIVE 1 JOIN

```
select customers_table.email  
  
from customers_table INNER JOIN address_table USING (address_id)  
  
where address_table.district ='Texas'
```

Derived Table Sub-queries

ALTERNATIVE 2 VIEW AND JOIN

```
create view texas(address_id) as select address_id from address_table
```

```
where address_table.district ='Texas'
```

```
select customers_table.email
```

```
from customers_table INNER JOIN  texas
```

```
on customers_table.address_id=texas.address_id
```

Expression Subqueries

Single-Value

- Example: Retrieve the maximum length of movie

```
SELECT MAX(movies_table.length)
```

```
FROM movies_table;
```

Expression Subqueries

Single-Value

- Use a subquery to match the max length and find movie title

```
SELECT movies_table.title, movies_table.length
FROM movies_table
WHERE movies_table.length = (
    SELECT MAX(movies_table.length)
    FROM movies_table);
```

Boolean Expression Sub-queries

- Used to check a query against some special functions that return a Boolean expression. The functions include...
 - **IN**
 - **EXISTS**
 - **ANY / ALL**

Boolean Expression Sub-queries

- **Example using IN** - Used to check against a set of values

```
SELECT customers_table.email
```

```
FROM customers_table
```

```
WHERE customers_table.address_id IN(10, 122, 310)
```

Boolean Expression Sub-queries

- **Example using IN** - Used to check against a set of values that come from a subquery

```
select customers_table.email  
from customers_table  
where customers_table.address_id IN(  
    select address_id  
    from address_table  
    where address_table.district = 'Texas');
```

Sub-queries (cont'd)

- Example **EXISTS** - Uses data from the outer query in the sub-query. Sometimes referred to as a '*correlated*' sub-query.

```
SELECT movies.movie_id
FROM movies
WHERE NOT EXISTS (
    SELECT *
    FROM watched
    WHERE watched_table.movie_id = movies.movie_id );
```

Thanks

- Edmund Nevin for earlier versions of these lecture notes
- References:

<https://www.mysqltutorial.org/>

<https://dev.mysql.com/>

<https://www.w3schools.com/>

<https://home.unicode.org/>



**Break
Back at 8pm
for Lab session**

**TU257
Information Systems
Marisa Llorens**